

Lab 01 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — "Een container is een kleine VM"

Antwoord. Fout op het essentiële punt: een container bevat **geen besturingssysteem** en heeft **geen eigen kernel**. Het is een proces van de host, geïsoleerd door *namespaces* en beperkt door *cgroups*, uitgevoerd door de kernel van de host.

Waarom. Een VM start een kernel, dan een `init`, dan tientallen systeemdiensten (logging, cron, SSH, netwerk...) nog voor je applicatie start: vandaar de seconden of minuten opstart en de GB's schijf. Een container start daar niets van: de kernel draait al, we vragen hem enkel namespaces aan te maken en **één** proces te starten. De opstartkost is die van een `fork` + `exec`, enkele milliseconden; de schijfkost is die van enkel de bibliotheken die de applicatie nodig heeft.

→ LINUX

`fork` dupliceert het huidige proces, `exec` vervangt de inhoud ervan door een ander programma. Zo wordt *elk* Linux-proces geboren, `podman run` inbegrepen: Podman dupliceert zichzelf, de kloon stapt in zijn namespaces en voert dan jouw commando uit. Een container wordt precies zo geboren als een `ls`.

Nuance. De intuïtie "lichte VM" is niet absurd voor een *gebruiker*: je krijgt wel degelijk een `/`, een `hostname`, een IP, een `root`. Ze wordt gevaarlijk zodra het over beveiliging gaat: waar de hypervisor van een VM een echte grens is, deelt een container de kernel — een kernelfout doorkruist die grens. En op Windows is de "lichtheid" relatief: er is wel degelijk een VM, WSL 2, maar één enkele voor al je containers.

Voorbeeld.

```
time podman run --rm alpine echo hallo      # ~0,3 s, grotendeels de pull/CLI
podman run -d nginx:alpine                 # ~64 MB image, PID zichtbaar op de host via ps
```

Vraag 2 — 250 MB "zonder OS"

Antwoord. De image bevat de **userland** van een distributie: `/bin/sh`, `libc`, `coreutils`, de PostgreSQL-binaries, de configuratie. Wat **ontbreekt**, is de **kernel** — en daarmee alles wat alleen bij het opstarten bestaat: bootloader, `initrd`, kernelmodules, `systemd`, stuurprogramma's, hardwarebeheer.

Waarom. Een Linux-programma roept de kernel aan via systeemaanroepen (`open`, `read`, `fork`). Het hoeft geen kernel mee te nemen, het moet er alleen een vinden: die van de host volstaat. De image levert dus alleen wat erboven ontbreekt.

Nuance. Daarom kunnen een "Alpine"- en een "Debian"-image naast elkaar draaien op dezelfde Ubuntu WSL: drie userlands, één kernel. En daarom draait een Linux-image niet op een Windows-kernel — vandaar WSL.

Voorbeeld.

```
podman run --rm alpine cat /etc/os-release | head -1 # NAME="Alpine Linux"
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # STRIKT dezelfde kernel
```

Vraag 3 — Twee nginx-containers, twee verschillende `ps`

Antwoord. De `pid`-namespace. Elke container krijgt zijn eigen PID-tabel: zijn hoofdproces krijgt daar nummer 1 en het kan geen enkele externe PID zien.

Waarom. Een proces zien is een voorwaarde om erop in te werken (`kill`, `/proc/<pid>`). Door de zichtbaarheid weg te nemen, neemt de kernel de facto het vermogen weg om langs die weg schade aan te richten. Op de host bestaan die processen wel degelijk, met echte PID's.

Nuance. Het is geen beveiliging "in de strikte zin" omdat het **geen ondoordringbare grens** is: het is een zichtbeperking, opgelegd door dezelfde kernel als die van de container. Een container gestart met `--pid=host` of `--privileged` krijgt het volledige zicht terug, en een kernelfout omzeilt het mechanisme. De namespace isoleert; hij verdedigt niet. In rootless-modus voegt de `user`-namespace een echte barrière van *rechten* toe bovenop die barrière van *zicht*.

Voorbeeld.

```
podman run -d --name web nginx:alpine
podman exec web ps -o pid,comm # PID 1 = nginx, dan zijn workers
ps -ef | grep -c "[n]ginx" # op de host: de processen zijn er wel degelijk
podman run --rm --pid=host alpine ps | head # isolatie weggenomen: de hele WSL
podman rm -f -t 0 web
```

Vraag 4 — Cannot connect to the Docker daemon

Antwoord. Bij je collega heeft de client gewerkt: hij toonde zijn versie, probeerde dan de socket `/var/run/docker.sock` te openen om de **server** te bevragen, en faalde. Twee waarschijnlijke oorzaken: (1) de daemon is niet gestart, (2) zijn gebruiker heeft geen rechten op de socket. Bij jou is die melding onmogelijk: Podman heeft **geen daemon** en contacteert geen enkele socket; elk commando doet het werk zelf, onder jouw gebruiker.

Waarom. Elk Docker-commando is een netwerkoproep naar `dockerd`. Het commando wijzigen verandert niets, want niemand heeft het nog gelezen: het probleem zit stroomopwaarts. Podman is een gewoon programma: als het start, werkt het.

Nuance. Er zijn twee gevallen waarin Podman wel een server heeft: `podman --remote` (of de variabele `CONTAINER_HOST`) dat met een `podman system service` op afstand praat, en `podman machine` onder Windows/macOS, waar de Windows-client met een VM praat. Je zou dan `unable to connect to Podman socket` zien. Onder WSL met Podman in Ubuntu geïnstalleerd, zit je in geen van beide gevallen.

Voorbeeld.

```
# Aan Docker-zijde, de diagnose:
systemctl status docker           # oorzaak 1: inactive (dead)
ls -l /var/run/docker.sock        # srw-rw---- root docker: je moet in de groep docker zitten
# Aan Podman-zijde, het bewijs dat er niets te contacteren valt:
podman version                    # één enkel blok "Client"
podman --remote version           # Error: unable to connect to Podman socket ...
```

Vraag 5 — "Een image draait niet"

Antwoord. Een image is een verzameling alleen-lezen bestanden plus metadata. Er is geen proces, geen toestand, niets om in te plannen: ze is even inert als een `.zip` met een handleiding erbij.

Waarom. Bij `podman run` voegt de engine drie dingen toe: (1) een dunne **schrijflaag** bovenop de alleen-lezen lagen, (2) een set **namespaces en cgroups**, (3) de **uitvoering** van het commando dat in de metadata van de image staat (`ENTRYPOINT` / `CMD`), via de runtime `crun`. Het resultaat van die assemblage is de container.

Nuance. De *aangemaakte* container en de *gestarte* container zijn twee aparte stappen: `podman create` doet alles behalve het proces starten, `podman start` start het. `podman run` is gewoon `create` + `start` (+ `pull` als de image ontbreekt).

Voorbeeld.

```
podman create --name tmp alpine sleep 30 # container aangemaakt, geen proces
podman ps -a --filter name=tmp           # STATUS: Created
podman start tmp && podman ps              # STATUS: Up -> nu draait hij
podman rm -f -t 0 tmp
```

Vraag 6 — PostgreSQL-gegevens na een `podman rm`

Antwoord. Nee, de gegevens zijn verdwenen. En nee, de image is **niet** gewijzigd: ze is strikt identiek voor en na.

Waarom. De schrijfacties van een container (via *copy-on-write*) komen terecht in zijn privé-schrijflaag. `podman rm` verwijdert de container **én** die laag. De lagen van de image zijn alleen-lezen: niets van wat een container doet, kan ze wijzigen — dat garandeert dat twee containers van dezelfde image vanuit dezelfde toestand vertrekken.

Nuance. Twee belangrijke correcties. Ten eerste vernietigt `podman stop` niets: een gestopte container behoudt zijn laag, en `podman start` vindt de gegevens terug. Het is wel degelijk `rm` dat vernietigt. Ten tweede declareert de officiële image `postgres` `/var/lib/postgresql/data` als `VOLUME`: de engine maakt dan een **anoniem** volume aan dat de `rm` overleeft — maar zonder naam is het bijna onmogelijk terug te vinden. In de praktijk beschouwt men de gegevens als verloren. Het benoemde volume is het onderwerp van lab 06.

Voorbeeld.

```
podman run -d --name c1 alpine sleep 600
podman exec c1 sh -c 'echo x > /merk.txt'
podman rm -f -t 0 c1
podman run --rm alpine ls /merk.txt      # No such file or directory: de image is intact
```

Vraag 7 — Tien containers, hoeveel schijf?

Antwoord. Enkele tientallen kilobytes in totaal — niet 10×210 MB. Elke container kost alleen zijn schrijfslag, aanvankelijk leeg, plus enkele configuratiebestanden (`hostname` , `resolv.conf` ...).

Waarom. De lagen van de image worden alleen-lezen **gedeeld** door alle containers die eruit voortkomen. Het opslagstuurprogramma `overlay` stapelt die lagen en één lege schrijfslag per container; een bestand wordt pas naar die laag gekopieerd op het moment dat het gewijzigd wordt (*copy-on-write*).

Nuance. Het antwoord verandert als elke container veel schrijft (logs, tijdelijke bestanden): elke wijziging van een imagebestand kopieert het volledig naar de laag van de container. En `podman ps -s` toont beide cijfers: de eigen grootte en de "virtuele" grootte.

Voorbeeld.

```
for i in 1 2 3; do podman run -d --name t$i alpine sleep 600; done
podman ps -s --format 'table {{.Names}}\t{{.Size}}'    # 11.4kB (virtual 8.72MB) elk
podman rm -f -t 0 t1 t2 t3
```

Vraag 8 — `root` in de container, `1000` op de host

Antwoord. De `user` -namespace projecteert de identificaties van de container op die van de host: UID 0 van de container *is* jouw UID 1000. Die "root" heeft, tegenover de kernel en de bestanden van de host, alleen jouw rechten: een poging om te schrijven in `/etc/shadow` , gemount vanaf de host, faalt met `Permission denied` , precies alsof je het zelf deed.

Waarom. De kernel controleert de rechten met de **echte** identiteit (hostzijde), niet met de identiteit die in de namespace getoond wordt. De UID's 1 tot 65536 van de container worden geprojecteerd op een gereserveerd bereik in `/etc/subuid` (`100000-165535`) dat geen enkel recht heeft op jouw bestanden.

Nuance. Die root *is* root **binnen** zijn namespaces: hij kan pakketten installeren, de rechten van imagebestanden wijzigen, luisteren op poort 80 van de container. Wat hij niet kan, is de grens oversteken. Praktisch gevolg: een bestand dat de container aanmaakt onder UID 999 (de gebruiker `postgres`) verschijnt op je host met UID 100998 — de klassieke *bind mount*-valkuil in rootless-modus (lab 06).

Voorbeeld.

```
podman top waker user,huser                # root / 1000
podman unshare cat /proc/self/uid_map        # 0 -> 1000 (1), 1 -> 100000 (65536)
podman run --rm -v /etc:/host alpine sh -c 'echo x >> /host/shadow' # Permission denied
```

Vraag 9 — De groep `docker` en `sudo`

Antwoord. Lid zijn van de groep `docker` geeft het recht om naar `/var/run/docker.sock` te schrijven, dus om eender wat te laten uitvoeren door een daemon die als **root** draait: `docker run -v /:/host --privileged` geeft de hele host. Dat is `sudo` zonder wachtwoord, zonder logging en zonder limiet. Met rootless Podman is er noch een root-daemon noch een socket: de gebruiker kan niets meer dan wat hij al kon, en de regel heeft geen voorwerp meer.

Waarom. Auditing vereist dat men weet *wie wat* gedaan heeft. Een commando dat via de socket passeert, wordt uitgevoerd door `dockerd`, onder de identiteit `root`, zonder spoor dat aan de gebruiker gekoppeld is. `sudo docker ...` laat tenminste een regel na in `auth.log`. Rootless Podman gaat verder: de container is een proces van de gebruiker, zichtbaar en toewijsbaar in `ps`.

Nuance. Rootless Podman heeft een prijs: geen poort < 1024 zonder instelling, een iets trager netwerk in userspace, bepaalde mounts en opties verboden. In productie kom je ook *rootful* Podman tegen (`sudo podman`), dat dan dezelfde voorzorgen als Docker terugbrengt.

Voorbeeld.

```
# Wat de groep docker toelaat (NIET DOEN op een gedeelde machine):
docker run --rm -v /:/host alpine cat /host/etc/shadow # leesbaar: de daemon is root
# Hetzelfde onder rootless Podman:
podman run --rm -v /:/host alpine cat /host/etc/shadow # Permission denied
```

Vraag 10 — Lange vorm, korte vorm

Antwoord.

Korte vorm	Lange vorm	Object
<code>podman ps -a</code>	<code>podman container ls -a</code>	container
<code>podman images</code>	<code>podman image ls</code>	image
<code>podman rmi nginx:alpine</code>	<code>podman image rm nginx:alpine</code>	image
<code>podman rm web</code>	<code>podman container rm web</code>	container

Waarom. `ps` en `images` dateren van de eerste Docker-versies (2013), toen de CLI nog geen objecten had: `ps` imiteerde het gelijknamige Unix-commando, `images` was een meervoud. De grammatica `object actie` kwam er in 2017 (Docker 1.13), en Podman nam ze ongewijzigd over. De korte vormen blijven bestaan om niets te breken.

Nuance. De lange vorm is de enige volledige: `podman container ls`, `podman image ls`, `podman volume ls`, `podman network ls`, `podman pod ls` volgen hetzelfde patroon, terwijl `podman ps` geen equivalent heeft voor volumes. In scripts: verkies de lange vorm.

Voorbeeld.

```
podman container ls -a --format '{{.Names}}'
podman image ls --format '{{.Repository}}:{{.Tag}}'
```

Vraag 11 — Twee keer `uname -r`, twee hosts

Antwoord. `uname -r` is een systeemaanroep: de waarde komt van de **kernel**, nooit van de image. Onder WSL is de kernel `microsoft-standard-WSL2`, door Microsoft gecompileerd voor de WSL-VM; op een native Ubuntu-server is het de `generic`-kernel uit het Ubuntu-pakket. Een container toont de kernel van de machine die hem uitvoert, welke image ook.

Waarom. De container is een proces van de hostkernel; er zit geen kernel in de image (vraag 2). Op Windows is die host niet Windows maar de WSL 2-VM.

Nuance. "Licht" blijft waar: de WSL-VM is **uniek**, één keer gestart, en gedeeld door al je containers; die blijven processen die in milliseconden starten. Wat niet meer waar is, is "helemaal geen VM". Praktische gevolgen: het beschikbare RAM is dat van WSL (`.wslconfig`), en Windows-bestanden (`/mnt/c/...`) gemount in een container zijn traag, omdat ze de grens VM ↔ Windows oversteken. Werk in het Linux-bestandssysteem (`~`).

Voorbeeld.

```
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # identiek
podman info --format '{{.Host.Kernel}} {{.Host.MemTotal}}' # het RAM zoals WSL het ziet
```

Vraag 12 — Oneindige lus en RAM

Antwoord. Nee: de `pid`-namespace verbergt alleen processen. Het mechanisme dat de burens beschermt, is de geheugen-**cgroup**, geactiveerd door `--memory`. Zonder limiet verbruikt de container al het beschikbare RAM; wanneer de kernel niets meer over heeft, doodt de **OOM killer** een proces naar keuze — niet noodzakelijk de schuldige.

Waarom. Namespaces en cgroups zijn twee onafhankelijke mechanismen: het ene isoleert het *zicht*, het andere begrenst het *verbruik*. Met `--memory=512m` veroorzaakt een overschrijding de dood van enkel het proces van de container (`Exited (137)`, `OOMKilled: true`), en de rest van de machine merkt er niets van.

Nuance. In rootless-modus is `--memory` alleen mogelijk als `systemd` de controller `memory` aan je gebruiker delegeert — wat op Ubuntu WSL het geval is zodra `systemd=true` geactiveerd is. En voor Java is een cgroup-limiet alleen nuttig als de JVM ze respecteert: sinds Java 10 leest ze de cgroup automatisch (`-XX:MaxRAMPercentage`), maar een `-Xmx` die met de hand te hoog is ingesteld, overschrijdt ze toch.

Voorbeeld.

```
podman run -d --name limiet --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limiet # MEM USAGE / LIMIT: ... / 134.2MB
podman inspect --format '{{.State.OOMKilled}}' limiet # false - voorlopig
podman rm -f -t 0 limiet
```

Vraag 13 — Een image promoveren zonder ze opnieuw te bouwen

Antwoord. Twee eigenschappen: **onveranderlijkheid** (een image geïdentificeerd door haar digest verandert nooit) en de **OCI-standaard** (Docker en Podman produceren en lezen exact hetzelfde formaat). Wat in acceptatie getest is, is bit voor bit wat naar productie gaat, welke engine ook. Bij elke stap opnieuw bouwen breekt die garantie: twee builds van dezelfde code geven niet noodzakelijk dezelfde image.

Waarom. Een build hangt af van het moment: `apt-get install` neemt de versie van de dag, `FROM eclipse-temurin:21-jre` volgt een bewegende tag, Maven lost versiebereiken op. Tussen de acceptatiebuild en de productiebuild kan een afhankelijkheid gewijzigd zijn — en de validatie in acceptatie is niets meer waard.

Nuance. Promotie gebeurt niet door `latest` opnieuw te taggen, maar door naar de **digest** te verwijzen (`api@sha256:...`) of naar een onveranderlijke tag (`api:1.4.2`). Lab 02 komt erop terug.

En de engine doet er weinig toe: een `podman pull` van een image die met `docker push` gepusht is, is een banaal geval.

Voorbeeld.

```
podman image inspect --format '{{.Digest}}' registry.intern/api:1.4.2
# Dezelfde digest op de werkpost (Podman), in acceptatie (Docker) en in productie (rootful Podman).
```

Vraag 14 — `alias docker=podman`

Antwoord. Zonder voorbehoud waar voor: (1) de hele image-cyclus — `build`, `pull`, `push`, `tag`, `images`, `history`, `inspect`, de Dockerfiles; (2) de levenscyclus van containers — `run`, `ps`, `logs`, `exec`, `stop`, `rm` met hun opties. Onwaar of anders: (a) **geen daemon** — geen `docker.sock`, `--restart=always` overleeft geen herstart zonder `systemd`, `podman rm -f` wacht 10 s; (b) **rootless** — geen poort < 1024 zonder instelling, IP-adressen afwezig met `pasta`, verschoven UID's op *bind mount*-bestanden, `--memory` afhankelijk van cgroup-delegatie.

Waarom. Podman heeft het oppervlak van Docker gekopieerd (de CLI, het formaat) maar niet zijn *architectuur*. Alles wat alleen van het oppervlak afhangt, is identiek; alles wat raakt aan "wie voert uit, met welke rechten, onder toezicht van wie" loopt uiteen.

Nuance. De verschillen zijn geen gebreken: elk is de tegenhanger van een veiligheidskeuze. En Docker Compose werkt met Podman (`podman compose`, lab 09), tegen de prijs van enkele configuratieregels.

Voorbeeld.

```
alias docker=podman
docker run -d --name web -p 8080:80 nginx:alpine      # identiek
docker run -d --name w80 -p 80:80 nginx:alpine      # Error: pasta failed ... Listen failed for
HOST TCP port */80: Permission denied
podman rm -f -t 0 web
```